Smart Home Monitoring System

IoT-Based Real-Time Environmental & Water-Tank Monitor

Platform	ESP32 + Arduino IDE
Protocol	MQTT over Wi-Fi (test.mosquitto.org)
Group	Group 10 | 46A 47A 48A 49A 50A
Date	March 2026

Group Members

SCS2315 – Electronics & Physical Computing | March 2026

Group 10 | 46A 47A 48A 49A 50A

Index No.	Full Name
23000821	R.K.K. Jinendra
23000481	G.C.K.S. Gajanayake
23000287	R.M.K.K.W. Deepal
23001161	M.M.M.K.B. Munasinghe
23000211	M.M.U. Brantha
23000058	M.N. Alwis
23001437	N.A.K.M. Perera
23000511	M.G.C.H.P. Gamage
23000317	K.K.N. Dilrukshi
23001593	A.G.G.G. Ransara

Table of Contents

1 Introduction **4**
1.1 Objectives . . . 4

2 Node Ecosystem & System Portability **4**
2.1 How the Three Core Nodes Work as an Ecosystem . . . 4
2.1.1 Step-by-Step Lifecycle . . . 5
2.2 System Portability . . . 6
2.2.1 Hardware Portability . . . 6
2.2.2 Software Portability . . . 6

3 System Architecture **6**
3.1 High-Level Overview . . . 6
3.2 MQTT Broker . . . 7
3.3 Node Summary . . . 7

4 Hardware **7**
4.1 Microcontroller: ESP32 . . . 8
4.2 Sensors and Actuators . . . 8
4.3 Wiring Notes . . . 8

5 Firmware – Node Descriptions **8**
5.1 Water-Tank Level Node (`ultra.ino` / `tank.ino`) . . . 8
5.1.1 Measurement . . . 9
5.1.2 Level Calculation . . . 9
5.1.3 Status Thresholds . . . 9
5.1.4 MQTT Payload (ultra.ino) . . . 9
5.2 Environment Node (`env.ino`) . . . 9
5.3 Buzzer Alert Node (`buzer.ino`) . . . 10
5.4 Button / Serial Node (`sketch_feb25a.ino`) . . . 10
5.5 DHT-Only Node (`dht_only.ino`) . . . 10

6 Software – Web Dashboards **10**
6.1 Central Dashboard (`central_dashboard.html`) . . . 11
6.2 Water-Tank Dashboard (`tank_dashboard.html`) . . . 11
6.3 Environment Dashboard (`env_dashboard.html`) . . . 11
6.4 MQTT Connection Lifecycle (Dashboards) . . . 11

7 Libraries and Dependencies **12**

8 Communication Flow **12**

9 Key Design Decisions **12**

10 Testing and Results **13**
10.1 Unit Tests 13
10.2 Integration Test 13
10.3 Limitations 13

11 Screenshots **14**

1 Introduction

The Internet of Things (IoT) enables physical devices to collect, exchange, and act on data over a network with minimal human intervention. This project applies those principles to a domestic setting, building a **Smart Home Monitoring System** that continuously tracks the water level inside a tank, records environmental conditions (temperature, humidity, ambient light), triggers an audible alert when the water runs low, and allows manual inputs via a push-button.

All nodes communicate wirelessly using the lightweight **MQTT** (Message Queuing Telemetry Transport) protocol through a public broker (`test.mosquitto.org`). A set of HTML dashboards running in any modern web browser subscribe to the same broker and display live metrics, providing a unified view of the home environment without requiring any dedicated server infrastructure.

1.1 Objectives

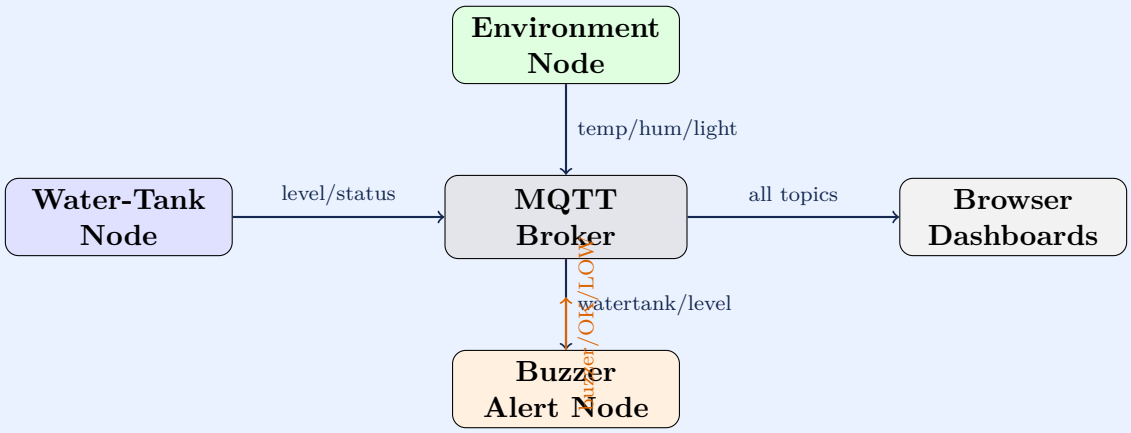
- Design and implement multiple ESP32 sensor nodes that publish data over Wi-Fi using MQTT.
- Build a modular architecture where nodes operate independently and can be added or removed without reconfiguring the rest of the system.
- Provide real-time, browser-based dashboards for each node and a unified Central Dashboard.
- Implement threshold-based alerting (low water level) using a buzzer node that subscribes to sensor data and reacts autonomously.
- Demonstrate proper use of IoT libraries: `WiFi`, `PubSubClient`, `DHT`, `NewPing`, and `ArduinoJson`.

2 Node Ecosystem & System Portability

2.1 How the Three Core Nodes Work as an Ecosystem

The three primary active nodes — the **Water-Tank Node**, the **Environment Node**, and the **Buzzer Alert Node** — are designed to act as a cohesive, self-organising ecosystem rather than three isolated sketches. Their interaction is orchestrated entirely by the MQTT broker: no node holds a direct reference to any other, yet together they deliver an end-to-end sense-react-alert loop.

Ecosystem Data-Flow



2.1.1 Step-by-Step Lifecycle

1. **Water-Tank Node publishes every 5 s.** It measures the water distance with the HC-SR04, calculates the fill percentage, and publishes a JSON payload to `home/watertank/level` along with a plain-text status (`FULL / NORMAL / LOW / CRITICAL`) to `home/watertank/status`.

2. **Environment Node publishes every 10 s.** Independently and concurrently, the Environment Node reads the DHT11 and LDR, then publishes four scalar values to `home/environment/*`. Its operation is completely decoupled from the tank node.

3. **Buzzer Node reacts in real time.** Because the Buzzer Node is subscribed to `home/watertank/level`, every tank publication triggers its MQTT callback within one broker round-trip (< 200 ms on a local Wi-Fi network). If the measured distance exceeds the `LOW_THRESHOLD_CM` (20 cm $\equiv$ tank below 25%), the buzzer fires its double-beep pattern and re-publishes `"LOW"` to `home/watertank/buzzer`; otherwise it publishes `"OK"`. *This creates a closed feedback loop entirely within the MQTT ecosystem* – no polling, no dedicated controller.

4. **Central Dashboard aggregates all streams.** The browser dashboard subscribes to every topic. As the three nodes publish at their own rates, the dashboard receives a continuous mixed stream and updates each card independently. The user sees a live unified view without any page refresh.

Ecosystem Properties

- **Asynchronous:** each node publishes on its own timer; no node waits for another.
- **Fault-tolerant:** if the Environment Node goes offline, the Water-Tank and Buzzer nodes continue operating unaffected.
- **Composable:** a new node (e.g. a PIR motion sensor) can join the ecosystem by simply publishing to a new topic – zero changes to existing firmware.
- **Observable:** any MQTT client (MQTT Explorer, Node-RED, another ESP32) can inspect the full state of the home at any time by subscribing to `home/#`.

2.2 System Portability

A key engineering goal was to make the system *portable* — deployable in any home, lab bench, or demonstration environment with minimal reconfiguration.

2.2.1 Hardware Portability

- **Self-contained nodes.** Each ESP32 node is a stand-alone unit powered via USB or a 5 V power bank. No wired interconnects exist between nodes; the only shared infrastructure is the Wi-Fi access point and the MQTT broker.
- **Single-file configuration.** Wi-Fi SSID/password and broker address are defined at the top of each sketch as `const char*` constants. Moving the system to a new network requires changing two lines and re-flashing – a two-minute operation.
- **GPIO abstraction.** All pin numbers are defined as named constants (`TRIG_PIN`, `DHT_PIN`, `BUZZER_PIN`, etc.), making it straightforward to adapt the wiring for a different ESP32 breakout board without editing business logic.
- **Power independence.** Because nodes communicate wirelessly they can be placed anywhere within Wi-Fi range: the tank node near the water storage, the environment node in a living area, and the buzzer node in a bedroom – each powered independently.

2.2.2 Software Portability

- **Broker-agnostic firmware.** Changing from the public `test.mosquitto.org` broker to a private home broker (e.g. a Raspberry Pi running Mosquitto) requires only updating the broker IP constant. No library changes, no topology changes.
- **Zero-install dashboards.** The HTML dashboards are single self-contained files. They can be opened from a USB drive, served from any web server, or hosted on GitHub Pages – no Node.js, no Python, no build step.
- **Platform independence.** The dashboards use the standard `mqtt.js` WebSocket client and plain HTML5/CSS3, making them compatible with any modern browser on any operating system (Windows, macOS, Linux, Android, iOS).
- **Scalability.** The MQTT topic hierarchy (`home/#`) can accommodate dozens of additional nodes without any schema migration. New sensor types simply publish under new sub-topics.

3 System Architecture

3.1 High-Level Overview

The system follows the classic IoT *Sense – Communicate – Visualise* pattern:

1. **Sense:** Each ESP32 node reads one or more physical sensors.
2. **Communicate:** Readings are serialised and published as MQTT messages to the public broker.

3. **Visualise:** HTML dashboards subscribe via MQTT-over-WebSocket and render the data in real time.
4. **React:** The Buzzer Node subscribes to the water-tank topic and acts on incoming data autonomously.

3.2 MQTT Broker

The project uses **test.mosquitto.org** (port 1883 for firmware, port 8081 WebSocket/TLS for browsers). This is a free, publicly accessible broker maintained by the Eclipse Mosquitto project, suitable for development and demonstration purposes.

MQTT Topic Hierarchy

```
home/
+-- environment/
|   +-- temperature       (deg C, float string)
|   +-- humidity          (%, float string)
|   +-- light             (raw ADC 0-4095)
|   \-- daynight          ("DAY" | "NIGHT")
+-- watertank/
|   +-- level             (JSON payload - see section 4.1)
|   +-- status            ("FULL" | "NORMAL" | "LOW" | "CRITICAL")
|   +-- distance          (cm, float string)
|   \-- buzzer            ("OK" | "LOW")
+-- button                (string - serial input)
+-- system/
|   \-- status            (online/offline beacons)
\-- alerts/
    \-- water             (JSON alert when CRITICAL)
```

3.3 Node Summary

Table 1: System Nodes

Node	Role	Key Topics
Ultra / Tank	Water-tank level (producer)	`home/watertank/level`, `home/watertank/status`
Env	Temperature, humidity, light (producer)	`home/environment/*`
Buzzer	Alert on low water (consumer + producer)	subscribes `home/watertank/level`; publishes `home/watertank/buzzer`
Button	Manual serial input (producer)	`home/button`
DHT Only	Standalone DHT11 demo (serial)	– (no Wi-Fi / MQTT)

4 Hardware

4.1 Microcontroller: ESP32

All firmware nodes run on the **ESP32** development board. The ESP32 is a dual-core 32-bit Tensilica LX6 processor with integrated Wi-Fi (802.11 b/g/n) and Bluetooth, making it well-suited for IoT applications. It operates at 3.3 V logic and provides multiple ADC channels, PWM outputs, and UART peripherals.

4.2 Sensors and Actuators

Table 2: Hardware Components

Component	Node	GPIO(s)	Description
HC-SR04 Ultrasonic	Tank / Ultra	TRIG: 14, ECHO: 12	Measures distance to water surface
DHT11 sensor	Env / DHT	4 (Env), 2 (DHT only)	Temperature & humidity
LDR (photoresistor)	Env	34 (ADC)	Ambient light level
Buzzer (active)	Buzzer	2	Audible low-water alert
Status LED	Tank, Ultra, Buzzer	4, 2, 4	Visual status indicator
Power LED	Tank, Ultra	16	Always-on power indicator
Push button	Button	via Serial	Manual MQTT publisher

4.3 Wiring Notes

- The HC-SR04 operates at 5 V; a resistor voltage divider (or logic-level shifter) on the ECHO line protects the ESP32's 3.3 V GPIO.
- The DHT11 data line requires a 10 kΩ pull-up resistor to 3.3 V.
- The LDR is wired as a voltage divider with a fixed 10 kΩ resistor; the junction feeds GPIO 34 (ADC channel 1).
- The active buzzer is driven directly from a GPIO through a small NPN transistor to avoid exceeding the ESP32's 12 mA per-pin current limit.

5 Firmware – Node Descriptions

5.1 Water-Tank Level Node (`ultra.ino` / `tank.ino`)

Two firmware variants implement the water-tank producer. `ultra.ino` is the full-featured version using the `NewPing` and `ArduinoJson` libraries; `tank.ino` is a simpler version with raw `pulseIn` timing.

5.1.1 Measurement

The HC-SR04 is triggered every **5 seconds**. The time-of-flight formula converts the echo pulse duration to distance:

$$d = \frac{t \cdot v_{\text{sound}}}{2} = \frac{t \times 0.0343\,\text{cm}/\mu\text{s}}{2}$$

where t is the echo duration in microseconds.

5.1.2 Level Calculation

Given a tank height of 100 cm, a sensor-to-empty-water distance of 90 cm, and a sensor-to-full-water distance of 10 cm:

$$\text{level}\% = \frac{d_{\text{empty}} - d_{\text{measured}}}{d_{\text{empty}} - d_{\text{full}}} \times 100$$

$$= \frac{90 - d}{90 - 10} \times 100 \qquad \text{clamped to } [0, 100]$$

5.1.3 Status Thresholds

Level (%)	Status
≥ 80	`FULL`
$50 \leq x < 80$	`NORMAL`
$20 \leq x < 50$	`LOW`
< 20	`CRITICAL`

5.1.4 MQTT Payload (ultra.ino)

Listing 1: Water-level JSON payload

```
{
  "node_id":      "WATER_TANK_01",
  "timestamp":    48320,
  "level_cm":     62.3,
  "percentage":   76.5,
  "distance_cm":  37.7,
  "status":       "NORMAL"
}
```

5.2 Environment Node (`env.ino`)

Reads DHT11 (temperature & humidity) and an LDR (ambient light) every **10 seconds** and publishes four separate scalar topics:

- `home/environment/temperature` – float string (°C)
- `home/environment/humidity` – float string (%)
- `home/environment/light` – raw ADC value

- `home/environment/daynight` – `"DAY"` or `"NIGHT"`

The day/night decision uses a configurable threshold of 20% relative light intensity:

Listing 2: Day/Night decision logic

```
float lightPct = (lightRaw / 4095.0) * 100.0;
const float DAY_THRESHOLD = 20.0;
const char* dayNight = (lightPct < DAY_THRESHOLD) ? "NIGHT" : "DAY";
```

5.3 Buzzer Alert Node (`buzer.ino`)

The buzzer node acts as a **pure consumer** of the water-tank data. On startup it subscribes to `home/watertank/level` and processes every incoming message in the MQTT callback:

Listing 3: Buzzer alert callback (abridged)

```
void onMessage(char* topic, byte* payload, unsigned int length) {
  float dist = atof(buf);          // parse distance value

  // Mirror distance for other subscribers
  client.publish(topic_dist, buf);

  if (dist >= LOW_THRESHOLD_CM) {  // LOW_THRESHOLD_CM = 20 cm
    digitalWrite(STATUS_LED, HIGH);
    alertBeep();                   // double-beep pattern
    client.publish(topic_buzzer, "LOW");
  } else {
    digitalWrite(BUZZER_PIN, LOW);
    client.publish(topic_buzzer, "OK");
  }
}
```

The `alertBeep()` function produces two short 200 ms pulses separated by a 150 ms pause, giving a distinctive double-beep pattern.

5.4 Button / Serial Node (`sketch_feb25a.ino`)

This simple node bridges the Arduino Serial Monitor to MQTT. Any line typed in the Serial Monitor is published verbatim to `home/button`. It is useful for manual testing and demonstrating the publish/subscribe model without physical sensors.

5.5 DHT-Only Node (`dht_only.ino`)

A standalone, offline DHT11 demonstration sketch. It reads temperature and humidity every 2 seconds and prints the values to the Serial Monitor. No Wi-Fi or MQTT is used; this node was used early in development to verify sensor wiring before integrating with the network stack.

6 Software – Web Dashboards

All dashboards are single-file HTML pages that require **no build step or server**. They load the `mqtt.js` client library from a CDN and connect directly to `test.mosquitto.org` over WebSocket (port 8081, TLS).

6.1 Central Dashboard (`central_dashboard.html`)

The Central Dashboard provides a *single-pane-of-glass* view of all nodes. It displays node cards for:

- [Water] Water Tank – level percentage and visual fill animation
- [Temp] Environment – temperature, humidity, light, day/night
- [Bell] Buzzer – current alert status (`OK` / `LOW`)
- [Button] Button – last message received on `home/button`

Each card includes an *Open* button that links to the dedicated node dashboard for deeper information.

6.2 Water-Tank Dashboard (`tank_dashboard.html`)

Renders an animated graphical tank that fills and drains in real time. The fill colour transitions from blue (normal) through amber (low) to red (critical). Numeric readouts show distance (cm), level (cm), and percentage.

6.3 Environment Dashboard (`env_dashboard.html`)

Displays four metric cards:

Card	Topic	Unit
Temperature	`home/environment/temperature`	°C
Humidity	`home/environment/humidity`	%
Light	`home/environment/light`	raw ADC
Day / Night	`home/environment/daynight`	DAY / NIGHT

6.4 MQTT Connection Lifecycle (Dashboards)

1. Page load triggers `mqtt.connect()` to the broker.
2. On `connect` event all relevant topics are subscribed.
3. On each `message` event the DOM is updated in place (no page reload).
4. A coloured badge indicates connection state: green (connected) / red (disconnected).
5. Automatic reconnect is attempted on `close` or `error` events.

7 Libraries and Dependencies

Table 3: Firmware Libraries

Library	Version	Purpose
`WiFi`	(built-in ESP32)	Wi-Fi station mode management
`PubSubClient`	≥ 2.8	MQTT client for Arduino/ESP32
`DHT`	Adafruit ≥ 1.4	DHT11/DHT22 sensor driver
`NewPing`	≥ 1.9	HC-SR04 ultrasonic driver with timeout
`ArduinoJson`	v7	JSON serialisation / deserialisation

Table 4: Dashboard Dependencies (CDN)

Library	Purpose
`mqtt.js` v4.3.7	MQTT client over WebSocket for browsers

8 Communication Flow

The diagram below (described textually) illustrates the message flow:

```
[HC-SR04]-->[Tank/Ultra ESP32]----PUBLISH level------>[MQTT Broker]
                                                       |
[DHT11 + LDR]-->[Env ESP32]------PUBLISH env data-->[MQTT Broker]
                                                       |
[Serial Input]-->[Button ESP32]--PUBLISH button------>[MQTT Broker]
                                                       |
             +-----------------------------------------------+
             |                                             |
SUBSCRIBE <-+<--- Buzzer ESP32 (subscribes watertank/level) |
             |     then publishes watertank/buzzer         |
             |                                             |
SUBSCRIBE <-+---- Browser Dashboards (all topics)
```

All ESP32 nodes reconnect automatically if the Wi-Fi or MQTT connection is dropped, using a non-blocking reconnect pattern based on `millis()` timestamps to avoid halting the main loop.

9 Key Design Decisions

1. **Decoupled publish/subscribe:** No node communicates directly with another. The broker acts as the sole intermediary, making it easy to add new subscribers (e.g. a logging service) without modifying existing firmware.
2. **Separate scalar topics vs. JSON:** The Environment node publishes each metric to its own topic, enabling lightweight subscribers to receive only the data they need. The Water-Tank node uses a single JSON payload to keep all related metrics atomic and consistent.

3. **Non-blocking reconnect:** Rather than using `while(!connected())` loops that would freeze the MCU, all nodes check the elapsed time with `millis()` and attempt reconnection only when a configurable interval has passed.
4. **Static web pages:** Hosting the dashboards as plain HTML files (no Node.js, no Flask) removes the need for a server and makes deployment trivial – open the file in a browser.
5. **Public broker for development:** Using `test.mosquitto.org` avoids the overhead of setting up a local broker during development and lab sessions. For a production deployment, a private broker with authentication would be used.

10 Testing and Results

10.1 Unit Tests

Each node was tested independently before integration:

- **DHT node:** Sensor readings verified via Serial Monitor; cross-checked with a reference thermometer.
- **Tank/Ultra node:** Distance measured at known positions (ruler); `level%` formula verified manually.
- **Buzzer node:** Behaviour confirmed by publishing test payloads via MQTT Explorer; beep triggered correctly at $d \geq 20\,\text{cm}$.
- **Button node:** Serial input forwarded to MQTT and received in browser dashboard.

10.2 Integration Test

All nodes were powered simultaneously. The Central Dashboard correctly displayed live readings from all four nodes. The buzzer fired within one MQTT round-trip (typically $<200\,\text{ms}$) of the tank node publishing a low-water reading.

10.3 Limitations

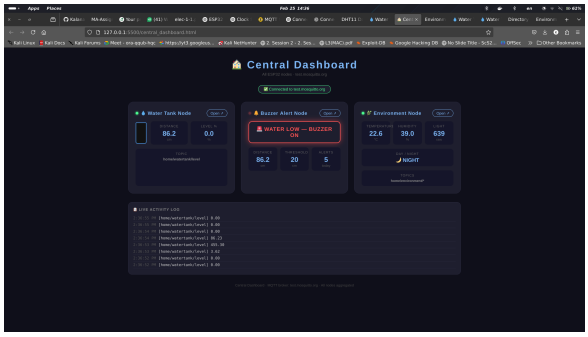
Known Limitations

- `test.mosquitto.org` is a public, unauthenticated broker. Any user on the internet can subscribe to the same topics. This is acceptable for a coursework demo but not for a real deployment.
- The DHT11 has $\pm 1\,°\text{C}$ temperature accuracy and $\pm 5\%$ humidity accuracy. A DHT22 would improve precision.
- Wi-Fi credentials are hard-coded in the firmware; a production system should use Wi-Fi provisioning or a secrets file.
- The single-echo HC-SR04 can give false readings if the water surface is turbulent; averaging several readings would improve stability.

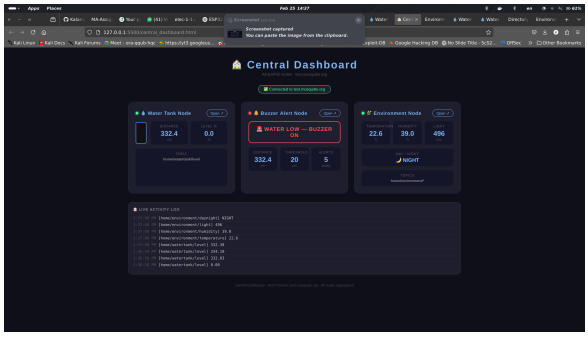
11 Screenshots

The following screenshots were captured during a live demonstration of the system on 25 February 2026, with all nodes running simultaneously. Hardware photos were taken on 6 March 2026.

Dashboard Screenshots – 25 February 2026

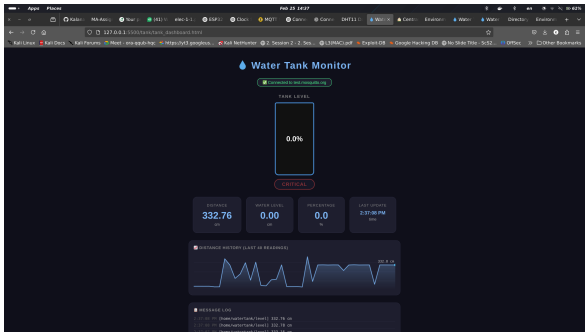


(a) 14:36:55



(b) 14:37:05

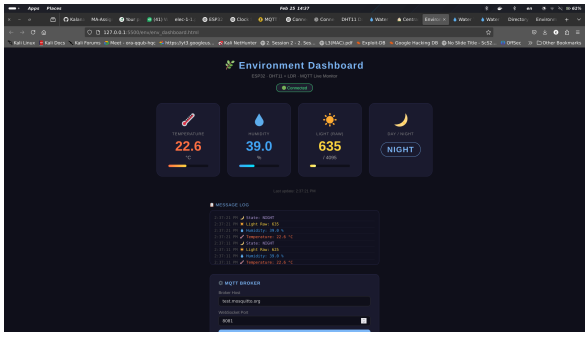
Figure 1: Live system – Screenshots 1 & 2



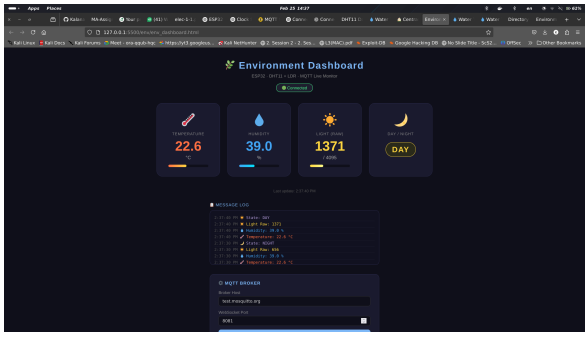
(a) 14:37:09

(b) 14:37:17

Figure 2: Live system – Screenshots 3 & 4



(a) 14:37:28



(b) 14:37:43

Figure 3: Live system – Screenshots 5 & 6

Hardware Photos – 6 March 2026

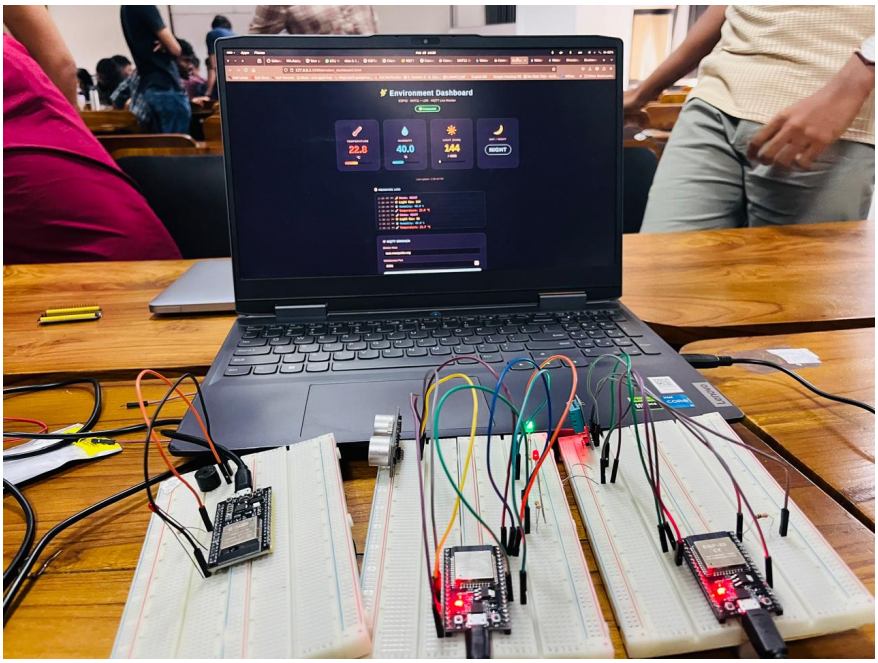
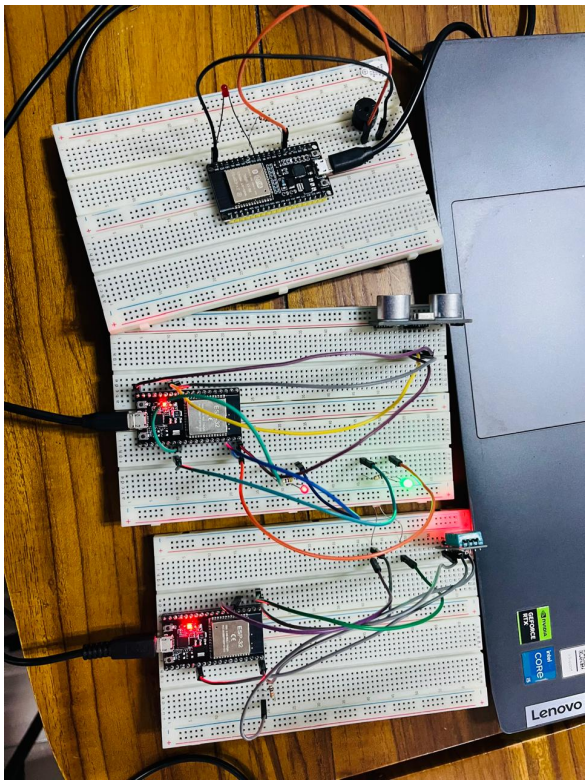
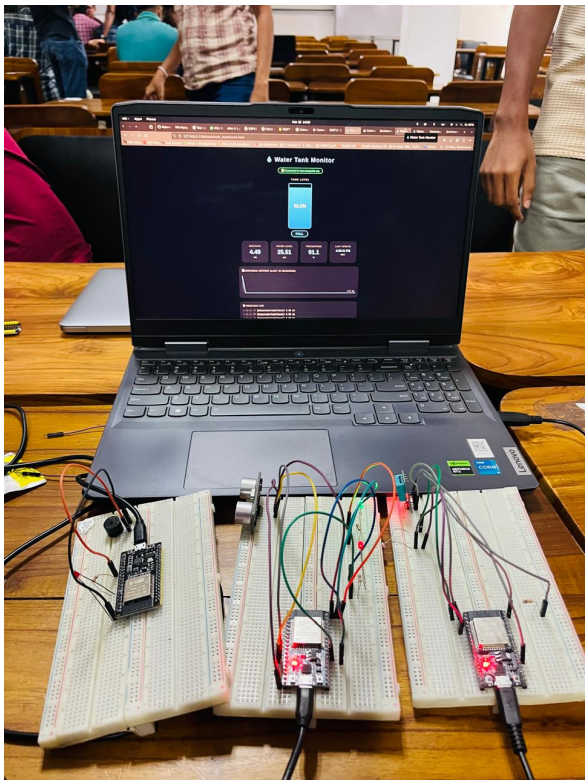


Figure 4: Hardware setup – Photo 1 (Group 10 demonstration)



(a) Hardware detail – Photo 2



(b) Hardware detail – Photo 3

Figure 5: Hardware photos – Group 10 sensor nodes

References

[1] OASIS Standard, *MQTT Version 5.0*, OASIS, 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] Eclipse Foundation, *Eclipse Mosquitto – An open source MQTT broker*. https://mosquitto.org

[3] N. O'Leary, *PubSubClient Arduino Library*, v2.8. https://pubsubclient.knolleary.net

[4] B. Beaupré, *ArduinoJson – A simple and efficient JSON library for embedded C++*, v7. https://arduinojson.org

[5] Espressif Systems, *ESP32 Series Datasheet*, v4.4. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[6] Aosong Electronics, *DHT11 Humidity & Temperature Sensor Datasheet*. https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *HC-SR04 Ultrasonic Distance Sensor Datasheet*. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf